HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

HỌC PHẦN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



BÁO CÁO BÀI TẬP(AGENT_01)

DISCOVERING AI AGENTS

TÌM HIỂU VỀ HỆ TÁC TỬ THÔNG MINH AI AGENT VÀ TÁC TỬ HỌC TĂNG CƯỜNG (REINFORCEMENT LEARNING)

Họ và tên:	Bùi Anh Đức
Lớp:	D23CTPM01
Mã sinh viên:	B23DCCN165
Học kỳ:	I năm học 2026 – 2027
Giảng viên hướng dẫn:	PGS.TS Trần Đình Quế

Hà Nội, tháng 10/2026

MỤC LỤC

1 Cơ sở lý thuyết về AI Agent **1**

1.1 Từ mô hình dự đoán đến tác tử thực hiện công việc 1

1.2 Đọc ba nguồn theo ba góc nhìn 2

1.2.1 Những khái niệm không nên gộp làm một 2

1.3 Mô tả tác tử bằng trạng thái, hành động và mục tiêu 3

1.4 Tác tử học và mối liên hệ với reinforcement learning 4

1.4.1 Khám phá và khai thác 4

1.4.2 Phạm vi của ví dụ bảng Q 4

1.5 Agent dùng mô hình ngôn ngữ và công cụ 5

1.5.1 Một tác tử hay nhiều tác tử 5

1.6 Đánh giá tác tử và liên hệ với phần thực hành 6

1.6.1 Điều kiện kiểm soát khi tác tử tự hoạt động 6

2 Thực hành Q-learning trong mê cung **7**

2.1 Bước 1 Chuẩn bị môi trường chạy 7

2.2 Bước 2 Xây dựng trạng thái và luật di chuyển 8

2.3 Bước 3 Hiển thị môi trường trước huấn luyện 9

2.4 Bước 4 Khởi tạo bảng Q và chính sách hành động 10

2.5 Bước 5 Huấn luyện và đọc output của cell train 11

2.5.1 Output thực tế của cell train 12

2.6 Bước 6 Theo dõi quá trình học bằng đồ thị 13

2.7 Bước 7 Đánh giá và đối chiếu đường ngắn nhất 15

2.7.1 Output của phép đánh giá 16

2.7.2 Đọc kết quả đánh giá cùng các điều kiện chạy 17

2.8 Bước 8 Kiểm tra seed và các điểm xuất phát khác 18

2.9 Bước 9 Demo tự hoạt động sau train 19

2.9.1 Output từng hành động của tác tử 20

2.9.2 Các thời điểm trong cùng một lượt tự chạy 21

2.10 Bước 10 Xuất demo và lưu kết quả 22

2.10.1 Tạo GIF từ rollout thực tế 22

2.10.2 Vẽ biểu đồ so sánh chính sách 22

2.10.3 Lưu log, cấu hình và số liệu 23

2.10.4 Chạy lại và chuẩn bị đưa lên GitHub 23

Tài liệu tham khảo **24**

Phần 1

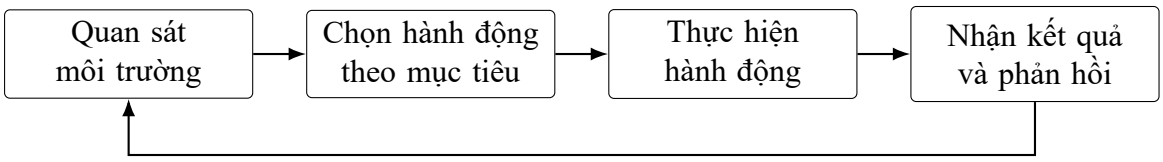
Cơ sở lý thuyết về AI Agent

1.1 Từ mô hình dự đoán đến tác tử thực hiện công việc

Một mô hình nhận dữ liệu đầu vào và trả về dự đoán chưa mô tả hết cách một hệ thống thông minh làm việc. Trong bài toán dự đoán giá nhà, đầu ra có thể là một con số. Nhưng với yêu cầu tìm căn nhà phù hợp, hệ thống còn phải xác định tiêu chí, lấy danh sách tin đăng, loại những căn không đáp ứng và kiểm tra lại thông tin thiếu. Mỗi kết quả trung gian có thể làm thay đổi bước xử lý tiếp theo. Đơn vị cần xem xét lúc này là cả quá trình ra quyết định và thực hiện hành động.

Google Cloud mô tả AI agent là hệ thống phần mềm dùng AI để theo đuổi mục tiêu thay người dùng, có khả năng lập luận, lập kế hoạch và ghi nhớ. IBM nhấn mạnh việc tự tổ chức công việc với các công cụ sẵn có. McKinsey đặt agent trong bối cảnh một thành phần phần mềm được giao quyền thực hiện nhiệm vụ cho người dùng hoặc hệ thống khác [1, 2, 3]. Ba cách diễn đạt cùng hướng đến quan hệ giữa mục tiêu, quyết định và hành động, nhưng chọn các góc nhìn khác nhau.

Cách hiểu được sử dụng trong bài: AI agent là một tác tử nhận thông tin từ môi trường, lựa chọn hành động để đạt mục tiêu, thực hiện hành động và sử dụng kết quả nhận được để quyết định bước tiếp theo. Tính tự chủ cần được đặt trong phạm vi đã giao: tác tử có thể tự chọn bước xử lý nhưng vẫn chịu các điều kiện về dữ liệu, công cụ và điểm dừng.



Hình 1.1: Vòng lặp quan sát và hành động. Sơ đồ tự xây dựng để mô tả cách hiểu về tác tử.

Điểm cần kiểm tra không chỉ là tác tử trả lời có hợp lý hay không, mà còn là môi trường đã thay đổi đúng yêu cầu hay chưa. Với ví dụ tìm nhà, danh sách phải thỏa điều kiện thực tế. Với tác tử trong mê cung, vị trí phải đi từ ô xuất phát đến đích bằng các bước hợp lệ. Một lời giải thích nghe thuyết phục không thay thế được bằng chứng hành động.

Từ cách hiểu trên, một bài thực hành nhỏ vẫn có thể biểu diễn được những thành phần cốt lõi của agent. Môi trường không nhất thiết là Internet; hành động không nhất thiết là gọi API. Một lưới ô vuông cùng bốn hướng di chuyển đã đủ để theo dõi quan sát, quyết định, phản hồi và kết quả cuối cùng.

1.2 Đọc ba nguồn theo ba góc nhìn

Bảng 1.1: Trọng tâm khi đọc các tài liệu được giao.

Nguồn	Trọng tâm nội dung	Câu hỏi rút ra khi thiết kế
Google Cloud [1]	Năng lực của tác tử: quan sát, lập luận, hành động, kế hoạch.	Hệ thống nhận thông tin và tác động vào môi trường qua đâu?
IBM [2]	Cách vận hành, công cụ và các dạng tác tử.	Cơ chế nào quyết định hành động tiếp theo?
McKinsey [3]	Tổ chức công việc, phối hợp và kiểm soát khi ứng dụng.	Công việc hoàn thành theo tiêu chí nào và ai chịu trách nhiệm?

Google Cloud giúp xác định các năng lực cần xem xét khi mô tả một agent [1]. Khi chuyển sang thiết kế bài toán, tôi tách các năng lực đó thành những câu hỏi có thể kiểm tra. Tác tử quan sát được toàn bộ dữ liệu hay chỉ một phần? Nó lưu lại thông tin nào? Hành động có làm thay đổi trạng thái bên ngoài hay chỉ tạo nội dung? Cách đặt câu hỏi này tránh việc liệt kê nhiều tính năng nhưng không chỉ ra chúng tham gia vào quyết định ra sao.

IBM phân biệt các dạng tác tử từ phản xạ đơn giản đến tác tử có khả năng học [2]. Ý nghĩa thực hành là chọn mức độ phức tạp theo bài toán. Với một điều kiện đã biết và phản ứng cố định, quy tắc có thể đủ dùng. Khi kết quả của hành động chỉ được biết sau nhiều bước thử, cần có cách đánh giá và điều chỉnh quyết định. Không có lý do đưa mô hình ngôn ngữ vào một bài toán chỉ gồm tọa độ và bốn hướng di chuyển nếu mục đích là quan sát quá trình học.

McKinsey bàn về agent ở quy mô quy trình và tổ chức công việc, bao gồm phân chia nhiệm vụ, phối hợp và phản hồi [3]. Từ góc nhìn này, một tác tử hoàn thành được một bước chưa đồng nghĩa toàn bộ công việc đã xong. Chẳng hạn, bước lấy dữ liệu có thể chạy thành công nhưng dữ liệu sai kỳ thời gian thì bước phân tích phía sau vẫn sai. Vì vậy, cần xác định cả kết quả trung gian và điều kiện chấp nhận đầu ra cuối cùng.

1.2.1 Những khái niệm không nên gộp làm một

Mô hình là một thành phần tính toán; agent còn bao gồm vòng điều khiển và cách tác động vào môi trường. Chatbot mô tả một hình thức tương tác bằng hội thoại. Việc giao tiếp qua chat chưa đủ để kết luận hệ thống có thể tự thực hiện một chuỗi công việc. Ngược lại, tác tử điều hướng có thể hoạt động mà không cần giao diện hội thoại.

Tương tự, tự chủ và học là hai trục khác nhau. Một chương trình có thể tự chạy theo chính sách cố định. Một mô hình có thể được train trước nhưng khi sử dụng chỉ thực hiện suy luận. Trong phần thực hành, quá trình cập nhật bảng giá trị được gọi là huấn luyện; giai đoạn dùng bảng đã lưu để đi đến đích là thực thi chính sách. Việc tách hai giai đoạn giúp xác định chính xác tác tử đang học hay đang sử dụng kết quả đã học.

1.3 Mô tả tác tử bằng trạng thái, hành động và mục tiêu

Để chuyển từ khái niệm sang chương trình, tôi dùng bài toán một robot đi trong lưới có vật cản. Robot bắt đầu ở ô s_0, mỗi lượt chọn trái, phải, lên hoặc xuống. Mục tiêu là đến ô đích với ít bước di chuyển và tránh va chạm. Đây là ví dụ minh họa tự xây dựng để phân tích thành phần hệ thống, không phải một tình huống triển khai doanh nghiệp trong ba bài đọc.

Bảng 1.2: Các thành phần cần xác định trước khi viết vòng lặp tác tử.

Thành phần	Cách xác định trong ví dụ điều hướng
Môi trường	Lưới cố định, gồm ô trống và ô vật cản.
Trạng thái	Tọa độ (r, c) hiện tại của robot.
Không gian hành động	Bốn phép dịch chuyển một ô theo trục hàng hoặc cột.
Phản hồi	Thưởng khi tới đích; phạt bước đi và hành động không hợp lệ.
Chính sách	Quy tắc chọn hành động từ trạng thái hiện tại.
Điểm dừng	Đến đích, va chạm hoặc hết giới hạn bước của một lượt.

Với bản đồ cố định và chuyển động không có quán tính, tọa độ hiện tại có thể được dùng làm trạng thái. Nếu thêm pin, cửa khóa hoặc vật cản thay đổi theo thời gian, cùng một tọa độ có thể dẫn tới những quyết định khác nhau. Khi đó cần bổ sung các biến liên quan. Chọn trạng thái quá ít làm mất thông tin; chọn quá nhiều lại khiến số trường hợp cần học tăng lên.

Chính sách được ký hiệu $\pi(a \mid s)$, biểu diễn xác suất chọn hành động a khi đang ở trạng thái s. Với chính sách xác định, tác tử luôn chọn cùng một hành động tại một trạng thái. Trong huấn luyện, chính sách có thể cố ý ngẫu nhiên để thử thêm lựa chọn. Trong demo sau train, có thể khóa việc khám phá để quan sát đường đi mà tác tử đã học.

Mục tiêu cũng cần chuyển thành số đo cụ thể. Chỉ đếm số lần tới đích sẽ bỏ qua trường hợp robot đi vòng rất lâu. Chỉ đếm số bước mà bỏ qua thất bại lại có thể khiến một lượt đâm tường ngay lập tức trông như có kết quả tốt. Vì vậy, tỷ lệ thành công, số bước và tổng reward phải được đọc cùng nhau.

Một agent cần có giới hạn thực thi. Nếu chính sách chọn lặp lại giữa hai ô, chương trình phải kết thúc lượt sau một số bước tối đa và ghi nhận việc chưa tới đích. Giới hạn này không tạo thêm năng lực cho tác tử; nó giữ cho quá trình chạy và đánh giá có điều kiện kết thúc rõ ràng. Đây cũng là lý do phải xem trạng thái kết thúc bên cạnh con số reward.

Bảng 1.3: Phân biệt năm dạng tác tử trong IBM [2] theo cơ chế quyết định.

Dạng tác tử	Điểm phân biệt
Phản xạ đơn giản	Chọn hành động theo điều kiện đang quan sát.
Phản xạ dựa trên mô hình	Dùng trạng thái nội bộ để bổ sung thông tin quá khứ.
Dựa trên mục tiêu	Xem xét chuỗi hành động có thể đưa tới đích.
Dựa trên độ hữu ích	So sánh mức lợi ích của các kết quả có thể đạt.
Có khả năng học	Điều chỉnh hành vi từ trải nghiệm và phản hồi.

1.4 Tác tử học và mối liên hệ với reinforcement learning

Trong học tăng cường, tác tử tương tác với môi trường và dùng reward để cải thiện cách chọn hành động [4]. Người lập trình xác định luật di chuyển và cách phản hồi, còn chính sách được hình thành thông qua trải nghiệm. Bài mê cung cho phép quan sát trực tiếp quá trình này vì trạng thái và hành động đều có số lượng nhỏ.

Một lượt chơi, hay episode, bắt đầu ở trạng thái xuất phát và kết thúc theo điều kiện đã đặt. Tại thời điểm t, tác tử quan sát s_t, chọn a_t, nhận r_{t+1} và chuyển sang s_{t+1}. Mục tiêu thường được biểu diễn bằng tổng reward có chiết khấu:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}, \qquad 0 \le \gamma < 1. \tag{1.1}$$

Hệ số γ quyết định mức ảnh hưởng của reward ở các bước sau. Trong mê cung, một hành động tốt có thể chưa mang lại thưởng ngay nhưng mở đường tới đích. Nếu chỉ nhìn reward tức thời, những ô chưa phải đích thường khó phân biệt với nhau.

Q-learning lưu giá trị $Q(s, a)$ cho từng cặp trạng thái và hành động. Với bước chuyển thông thường, phép cập nhật là:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]. \tag{1.2}$$

Trong đó α là tốc độ học. Khi chuyển sang trạng thái kết thúc, phần giá trị tương lai bằng 0. Cần xử lý riêng trường hợp này để không cộng thêm một giá trị không thuộc episode đang xét.

1.4.1 Khám phá và khai thác

Nếu luôn chọn giá trị Q lớn nhất ngay từ đầu, tác tử dễ lặp lại những lựa chọn đầu tiên khi bảng Q vẫn chưa chứa thông tin hữu ích. Chính sách ϵ-greedy dành xác suất ϵ để chọn ngẫu nhiên và xác suất còn lại để dùng hành động có Q cao nhất. Giảm ϵ theo episode giúp chuyển dần từ thử nghiệm sang sử dụng kinh nghiệm.

Việc giảm khám phá không phải bằng chứng tự động của việc học tốt. Nếu reward không tăng hoặc chính sách vẫn không tới đích, giảm ϵ chỉ làm tác tử lặp lại sai lầm ổn định hơn. Cần kiểm tra rollout không khám phá sau từng giai đoạn và so với một chính sách chưa học.

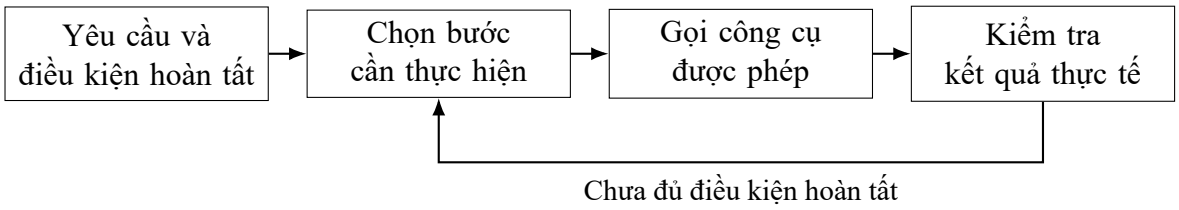
1.4.2 Phạm vi của ví dụ bảng Q

Mê cung nhỏ phù hợp với bảng tra cứu vì có thể lưu riêng bốn giá trị tại mỗi ô. Nếu trạng thái là ảnh camera hoặc câu hội thoại, cách liệt kê này không còn phù hợp. Khi đó phải thay đổi cách biểu diễn và phương pháp học. Kết quả tốt trên một bản đồ cố định chỉ xác nhận khả năng giải môi trường đó, chưa đủ để suy ra khả năng thích nghi trên bản đồ mới.

1.5 Agent dùng mô hình ngôn ngữ và công cụ

Trong hệ thống làm việc với văn bản, mô hình ngôn ngữ có thể tham gia diễn giải yêu cầu và lựa chọn công cụ. Nghiên cứu ReAct kết hợp quá trình suy luận với hành động và quan sát kết quả từ môi trường [5]. Điều cần rút ra khi thiết kế phần mềm là mỗi lần gọi công cụ phải đem lại dữ liệu để cập nhật trạng thái công việc, thay vì chỉ tạo ra một danh sách hành động có vẻ hợp lý.

Xét ví dụ tự xây dựng: người dùng yêu cầu tìm bài Assignment có notebook nhưng thiếu ảnh kết quả. Tác tử có thể liệt kê tệp, đọc metadata của notebook, kiểm tra các ô output, đối chiếu đường dẫn ảnh trong báo cáo rồi trả danh sách còn thiếu. Nếu kết quả kiểm tra cho biết ảnh được nhúng trực tiếp trong notebook, bước kiểm tra phải nhận diện trường hợp đó trước khi kết luận ảnh bị thiếu.



Hình 1.2: Vòng điều khiển minh họa cho công việc kiểm tra tệp bằng công cụ.

Ở ví dụ trên, phần mềm cần lưu những gì đã kiểm tra, kết quả của từng công cụ và các việc còn thiếu. Bộ nhớ phục vụ duy trì ngữ cảnh công việc. Việc thêm một kết quả mới vào ngữ cảnh không đồng nghĩa với cập nhật trọng số của mô hình. Nếu gọi cả hai quá trình là "học" mà không giải thích, sẽ khó biết hệ thống cải thiện nhờ dữ liệu đang nhìn thấy hay nhờ một lần train mới.

1.5.1 Một tác tử hay nhiều tác tử

Tôi chỉ tách công việc thành nhiều tác tử khi mỗi phần có đầu vào, đầu ra và tiêu chí đánh giá đủ rõ. Chẳng hạn, một thành phần kiểm tra code có thể trả danh sách cell lỗi; một thành phần kiểm tra tài liệu trả các hình chưa được tham chiếu. Thành phần điều phối phải đối chiếu các kết quả trước khi kết luận. Nếu tất cả cùng đọc một dữ liệu thiếu và cùng bỏ qua một điều kiện, việc có nhiều tác tử không sửa được lỗi gốc.

Với mê cung nhỏ, một tác tử là đủ. Thêm bước hội thoại hay điều phối không giúp kiểm chứng phép cập nhật Q. Lựa chọn này giữ mối quan hệ giữa kiến thức và kết quả chạy dễ theo dõi: hành động đến từ bảng Q, phản hồi đến từ môi trường, và đường đi demo là chuỗi quyết định thật của chính sách đã học.

1.6 Đánh giá tác tử và liên hệ với phần thực hành

Một tác tử cần được đánh giá theo nhiệm vụ đã giao. Với môi trường điều hướng, tôi dùng bốn câu hỏi: có đến đích không, mất bao nhiêu bước, tổng reward là bao nhiêu và kết quả có lặp lại được không. Các câu hỏi này xác định dữ liệu cần ghi ngay từ lúc viết vòng train, thay vì chỉ nhìn một ảnh đường đi sau cùng.

Bảng 1.4: Cách đọc các bằng chứng trong thực nghiệm điều hướng.

Bằng chứng	Kết luận có thể kiểm tra
Log của cell train	Episode thực sự đã chạy và các thống kê thay đổi theo thời gian.
Đường reward và số bước	Xu hướng của hành vi trong điều kiện có khám phá.
Đánh giá với $\epsilon = 0$	Năng lực của chính sách khi dùng trực tiếp bảng Q.
Đối chiếu đường ngắn nhất	Khoảng cách giữa đường học được và nghiệm tối ưu trên lưới.
Các khung hình demo	Thứ tự vị trí robot tự chọn khi thực thi.
Notebook có output	Mã, cấu hình và bằng chứng chạy được giữ cùng một nơi.

Một đường reward chưa phẳng không nhất thiết là lỗi huấn luyện: tác tử vẫn có thể ngẫu nhiên chọn hành động không tốt khi $\epsilon > 0$. Ngược lại, một rollout đẹp cũng chưa đủ nếu đó chỉ là lượt được chọn ra giữa nhiều thất bại. Vì vậy, đồ thị train cần đi cùng quy trình đánh giá đã nêu số lượt, seed, giới hạn bước và chính sách đang sử dụng.

Trong mê cung tất định, nhiều lần chạy một chính sách cố định từ cùng một điểm chỉ lặp lại cùng một bài kiểm tra. Nếu kết quả là 100%, con số đó không chứng minh tính tổng quát. Có thể kiểm tra thêm nhiều seed huấn luyện hoặc các trạng thái xuất phát khác, nhưng phải phân biệt chúng với bản đồ mới. Các nhận xét về độ ổn định cần giới hạn đúng theo những trường hợp thực sự đã chạy.

1.6.1 Điều kiện kiểm soát khi tác tử tự hoạt động

Khi tự xây dựng một tác tử, tôi đặt giới hạn số bước và lưu hành động để truy lại nguyên nhân thất bại. Với công việc trên tệp hoặc dịch vụ, có thể áp dụng cùng nguyên tắc bằng giới hạn số lần gọi công cụ và kiểm tra kết quả sau mỗi thao tác. Quyền thực hiện hành động phải gắn với nhiệm vụ cụ thể. Không thể lấy việc trả lời đúng một câu hỏi làm bằng chứng cho quyền thay đổi dữ liệu liên quan.

Phần thực hành tiếp theo tập trung vào learning agent trong một môi trường nhỏ có thể kiểm tra toàn bộ. Bảng Q đóng vai trò kinh nghiệm đã học; chính sách chọn hành động từ bảng này; reward cung cấp phản hồi. Kết quả cần đạt là robot tự đi từ điểm đầu đến đích bằng chính sách sau train, đồng thời notebook giữ đủ code, output và dữ liệu để kiểm tra lại từng bước.

Phần 2

Thực hành Q-learning trong mê cung

2.1 Bước 1 Chuẩn bị môi trường chạy

Bài toán thực hành sử dụng mê cung ở nguồn [4], gắn với learning agent trong cách phân loại của IBM [2]. Mã được viết lại trong notebook `AGENT_01_Q_Learning_Maze.ipynb`. Robot học bằng tương tác với môi trường; không dùng mô hình ngôn ngữ, dữ liệu gán nhãn đường đi hoặc API bên ngoài.

Trong cài đặt, chọn hành động là phần thực thi; cập nhật Q là phần học; reward cung cấp tín hiệu đánh giá; nhánh ngẫu nhiên của ϵ-greedy tạo cơ hội khám phá. Đây là cách liên hệ các thành phần learning agent của IBM với code cụ thể [2].

NumPy lưu bảng Q và tạo số ngẫu nhiên. Matplotlib vẽ mê cung, đồ thị; Pillow ghép các khung hình thành GIF. Thư mục `artifacts/` lưu bảng Q, log dạng CSV, ảnh và số liệu JSON. Seed của lượt train chính là 42. Bộ sinh số của đánh giá được tạo riêng để không làm thay đổi lịch khám phá trong huấn luyện.

In [1]:
```
from pathlib import Path
from collections import deque
import csv
import json
import platform
import numpy as np
import matplotlib
matplotlib.use("module://matplotlib_inline.backend_inline")
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from PIL import Image
from IPython.display import display, HTML

SEED = 42
ART = Path("artifacts")
ART.mkdir(exist_ok=True)
plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 11})
print(f"Python {platform.python_version()}")
print(f"NumPy {np.__version__} | Matplotlib {matplotlib.__version__}")
print(f"Training seed: {SEED}")
```

```
Python 3.12.14
NumPy 2.3.5 | Matplotlib 3.11.2
Training seed: 42
```

Hình 2.1: Cell chuẩn bị thư viện và output phiên bản môi trường chạy.

Ảnh code trong phần này được chụp từ bản HTML của notebook đã thực thi, giữ số `In` và output dạng văn bản. Các output đồ thị được đặt thành hình riêng ở bước tương ứng để đọc rõ hơn. Code được giữ đầy đủ trong notebook, có thể chạy lại theo thứ tự từ trên xuống.

2.2 Bước 2 Xây dựng trạng thái và luật di chuyển

Mảng `maze` có kích thước 10×10, gồm 43 ô trống và 57 ô vật cản. Điểm xuất phát là $(0, 0)$, đích là $(9, 9)$. Thứ tự hành động trong `actions` là trái, phải, lên, xuống. Hàm `valid` kiểm tra tọa độ có nằm trong bản đồ và có phải ô trống hay không.

In [2]:
```
maze = np.array([
    [0,1,1,1,1,1,1,1,1,1],
    [0,0,0,0,1,0,0,0,0,1],
    [1,1,1,0,1,0,1,1,0,1],
    [1,0,0,0,0,0,1,0,0,1],
    [1,0,1,1,1,1,1,0,1,1],
    [1,0,1,0,0,0,0,0,1,1],
    [1,0,1,0,1,1,1,0,1,1],
    [1,0,1,0,1,0,0,0,1,1],
    [1,0,1,0,1,0,1,0,0,1],
    [1,1,1,0,1,1,1,1,0,0],
], dtype=np.int8)
start, goal = (0, 0), (9, 9)
actions = ((0,-1), (0,1), (-1,0), (1,0))
action_names = ("LEFT", "RIGHT", "UP", "DOWN")
MAX_STEPS = 400

def valid(state):
    r, c = state
    return 0 <= r < 10 and 0 <= c < 10 and maze[r, c] == 0

def step(state, action):
    dr, dc = actions[action]
    nxt = (state[0] + dr, state[1] + dc)
    if not valid(nxt):
        return state, -10, True, "collision"
    if nxt == goal:
        return nxt, 50, True, "goal"
    return nxt, -1, False, "running"

print(f"Maze: {maze.shape}, free cells: {(maze == 0).sum()}")
print(f"Start: {start}, goal: {goal}, max steps: {MAX_STEPS}")
print("Rewards: step=-1, collision=-10, goal=+50")
```

```
Maze: (10, 10), free cells: 43
Start: (0, 0), goal: (9, 9), max steps: 400
Rewards: step=-1, collision=-10, goal=+50
```

Hình 2.2: Khai báo bản đồ, hàm chuyển trạng thái và output kiểm tra môi trường.

Hàm `step` là nơi duy nhất áp dụng luật môi trường. Nếu hành động không hợp lệ, robot giữ vị trí hiện tại, nhận -10 và kết thúc lượt. Nếu tới đích, reward là $+50$ và lượt cũng kết thúc. Bước hợp lệ chưa tới đích nhận -1. Giới hạn 400 hành động chặn trường hợp đi lặp. Cách trả về đồng thời vị trí mới, reward, cờ kết thúc và trạng thái giúp kiểm tra nguyên nhân dừng của mỗi episode.

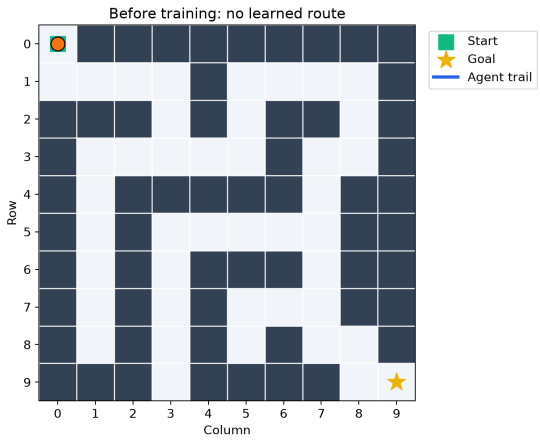
2.3 Bước 3 Hiển thị môi trường trước huấn luyện

Hàm `draw_maze` chỉ vẽ trạng thái và dấu vết được truyền vào. Nó không tìm đường hoặc thay đổi bảng Q. Chỉ số hàng tăng theo chiều từ trên xuống, phù hợp cách truy cập phần tử của NumPy. Robot màu cam bắt đầu ở ô xanh; đích được đánh dấu bằng ngôi sao.

```
In [3]: def draw_maze(ax, path=None, title="Maze environment"):
            ax.imshow(maze, cmap=ListedColormap(["#f1f5f9", "#334155"]))
            ax.set_xticks(range(10)); ax.set_yticks(range(10))
            ax.set_xticks(np.arange(-0.5, 10, 1), minor=True)
            ax.set_yticks(np.arange(-0.5, 10, 1), minor=True)
            ax.grid(which="minor", color="white", linewidth=1)
            ax.tick_params(which="minor", bottom=False, left=False)
            ax.scatter(start[1], start[0], s=180, c="#10b981", marker="s",
        label="Start")
            ax.scatter(goal[1], goal[0], s=280, c="#eab308", marker="*", label="Goal")
            if path:
                rr, cc = zip(*path)
                ax.plot(cc, rr, color="#2563eb", linewidth=3, label="Agent trail")
                ax.scatter(cc[-1], rr[-1], s=160, c="#f97316", edgecolors="black",
        zorder=5)
            ax.set_xlabel("Column"); ax.set_ylabel("Row")
            ax.set_title(title)

        fig, ax = plt.subplots(figsize=(6.2, 6.2))
        draw_maze(ax, [start], "Before training: no learned route")
        ax.legend(loc="upper left", bbox_to_anchor=(1.02, 1))
        fig.savefig(ART / "maze_before.png", dpi=180, bbox_inches="tight")
        plt.show()
```

Hình 2.3: Hàm vẽ mê cung và lưu ảnh trạng thái ban đầu trong notebook.



Hình 2.4: Mê cung trước train. Tác tử chưa có đường đi đã học.

Ở thời điểm này, thông tin được lập trình sẵn là bản đồ và luật phản hồi. Không có chuỗi vị trí dẫn tới đích được đưa vào hàm chọn hành động. Đường đi chỉ xuất hiện sau khi chính sách được train và thực thi.

2.4 Bước 4 Khởi tạo bảng Q và chính sách hành động

Bảng Q có hình dạng $10 \times 10 \times 4$, tương ứng 400 giá trị. Bốn giá trị tại mỗi ô ứng với bốn hành động. Các ô vật cản vẫn có chỗ trong mảng để việc đánh chỉ số đơn giản, nhưng không được dùng làm trạng thái hợp lệ. Tất cả giá trị được khởi tạo bằng 0.

In [4]:
```
EPISODES = 5000
ALPHA, GAMMA = 0.1, 0.9
EPS_START, EPS_MIN, EPS_DECAY = 0.5, 0.01, 0.995

def choose_action(q, state, epsilon, rng):
    if rng.random() < epsilon:
        return int(rng.integers(4))
    values = q[state]
    ties = np.flatnonzero(values == values.max())
    return int(rng.choice(ties))

def rollout(q, random_policy=False, seed=10000, initial=start):
    rng = np.random.default_rng(seed)
    state, path, trace, total = initial, [initial], [], 0
    for t in range(1, MAX_STEPS + 1):
        a = int(rng.integers(4)) if random_policy else
int(np.argmax(q[state]))
        nxt, reward, done, status = step(state, a)
        trace.append((t, state, action_names[a], nxt, reward, status))
        path.append(nxt)
        total += reward
        state = nxt
        if done:
            return path, trace, total, status
    return path, trace, total, "timeout"

q_initial = np.zeros(maze.shape + (4,), dtype=np.float64)
print(f"Q-table: {q_initial.shape}, entries: {q_initial.size}")
print(f"Episodes={EPISODES}, alpha={ALPHA}, gamma={GAMMA}")
print(f"Epsilon: {EPS_START} -> {EPS_MIN}, decay={EPS_DECAY}")
```

```
Q-table: (10, 10, 4), entries: 400
Episodes=5000, alpha=0.1, gamma=0.9
Epsilon: 0.5 -> 0.01, decay=0.995
```

Hình 2.5: Thông số Q-learning, cách chọn hành động và hàm thực thi chính sách.

Hàm `choose_action` dùng ϵ-greedy. Trong train, khi nhiều hành động có giá trị Q bằng nhau, chương trình chọn ngẫu nhiên giữa các hành động đó để tránh thiên lệch do thứ tự trong mảng. Khi demo, `rollout` chọn trực tiếp `argmax` và không khám phá.

`rollout` không loại hành động đâm tường, không cấm quay lại ô cũ và không nhận đường đi từ BFS. Nếu Q học chưa tốt, lỗi vẫn xuất hiện trong kết quả. Cách này kiểm tra hành vi thật của chính sách, tránh việc một bộ lọc tìm đường che đi các quyết định sai.

2.5 Bước 5 Huấn luyện và đọc output của cell train

Lượt train chính gồm 5.000 episode, $\alpha = 0.1$, $\gamma = 0.9$. Epsilon bắt đầu ở 0.5, nhân với 0.995 sau mỗi episode và không nhỏ hơn 0.01. Mỗi lượt bắt đầu lại ở $(0, 0)$; chương trình ghi reward, số bước, kết quả và epsilon vào `history`.

In [5]:
```
def train(seed, verbose=False):
    rng = np.random.default_rng(seed)
    q = np.zeros(maze.shape + (4,), dtype=np.float64)
    epsilon, history = EPS_START, []
    for episode in range(1, EPISODES + 1):
        state, total = start, 0
        epsilon_used = epsilon
        for t in range(1, MAX_STEPS + 1):
            a = choose_action(q, state, epsilon, rng)
            nxt, reward, done, status = step(state, a)
            terminal = done or t == MAX_STEPS
            target = reward if terminal else reward + GAMMA * q[nxt].max()
            q[state][a] += ALPHA * (target - q[state][a])
            state, total = nxt, total + reward
            if terminal:
                status = status if done else "timeout"
                break
        history.append({"episode": episode, "reward": total,
                        "steps": t, "success": int(status == "goal"),
                        "epsilon": epsilon_used, "status": status})
        epsilon = max(EPS_MIN, epsilon * EPS_DECAY)
        if verbose and episode % 500 == 0:
            block = history[-500:]
            avg_r = np.mean([x["reward"] for x in block])
            avg_s = np.mean([x["steps"] for x in block])
            rate = np.mean([x["success"] for x in block])
            print(f"Episode {episode:4d} | reward={avg_r:7.2f} | "
                  f"steps={avg_s:6.2f} | success={rate:6.1%} | eps_next=
{epsilon:.4f}")
    return q, history

q, history = train(SEED, verbose=True)
np.save(ART / "q_table.npy", q)
with (ART / "training_history.csv").open("w", newline="", encoding="utf-8") as
f:
    writer = csv.DictWriter(f, fieldnames=history[0].keys())
    writer.writeheader()
    writer.writerows(history)
print("Saved: artifacts/q_table.npy and training_history.csv")
```

Hình 2.6: Toàn bộ mã vòng train, ghi log và lưu bảng Q trong cell 5.

2.5.1 Output thực tế của cell train

```
Episode  500 | reward= -20.10 | steps= 11.10 | success=  0.0% | eps_next=0.0408
Episode 1000 | reward=  -4.15 | steps= 21.91 | success= 44.6% | eps_next=0.0100
Episode 1500 | reward=  20.27 | steps= 22.33 | success= 86.0% | eps_next=0.0100
Episode 2000 | reward=  20.65 | steps= 22.31 | success= 86.6% | eps_next=0.0100
Episode 2500 | reward=  21.37 | steps= 23.03 | success= 89.0% | eps_next=0.0100
Episode 3000 | reward=  20.74 | steps= 22.34 | success= 86.8% | eps_next=0.0100
Episode 3500 | reward=  21.81 | steps= 22.95 | success= 89.6% | eps_next=0.0100
Episode 4000 | reward=  20.48 | steps= 22.36 | success= 86.4% | eps_next=0.0100
Episode 4500 | reward=  20.87 | steps= 22.57 | success= 87.4% | eps_next=0.0100
Episode 5000 | reward=  20.94 | steps= 22.62 | success= 87.6% | eps_next=0.0100
Saved: artifacts/q_table.npy and training_history.csv
```

Hình 2.7: Output đầy đủ của cell 5, in thống kê sau mỗi 500 episode.

Trong phép cập nhật, `target` bằng reward nếu lượt kết thúc; các bước còn lại cộng thêm $\gamma \max Q(s', a')$. Dòng cập nhật `q[state][a]` đưa một phần chênh lệch giữa target và giá trị cũ vào bảng Q. Bước cuối chạm giới hạn 400 cũng được xem là kết thúc của lượt hữu hạn trong cài đặt này.

Mỗi dòng log là trung bình của 500 episode vừa chạy. `eps_next` là epsilon của episode kế tiếp. Sau train, chương trình lưu `q_table.npy` và `training_history.csv`; các tệp này do chính cell trên tạo ra.

2.6 Bước 6 Theo dõi quá trình học bằng đồ thị

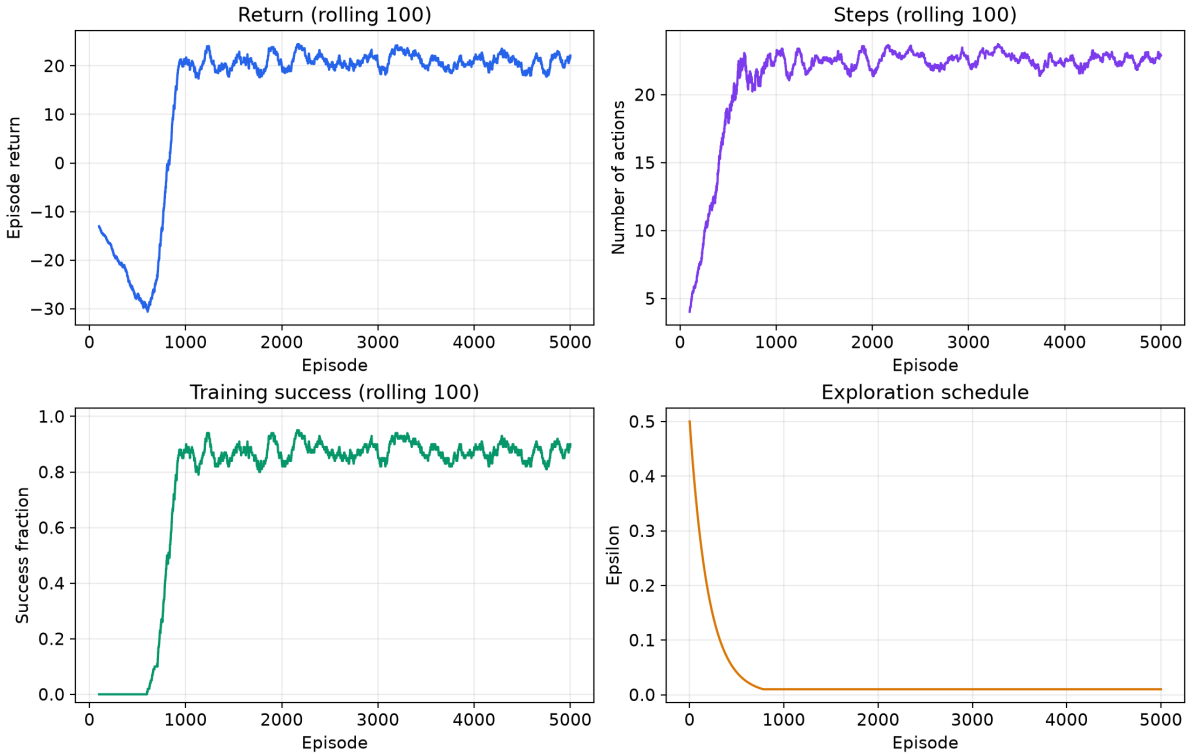
Cell tiếp theo đọc trực tiếp `history` để vẽ bốn đồ thị. Reward, số bước và tỷ lệ thành công dùng trung bình trượt 100 episode. Đường epsilon thể hiện giá trị thực ở từng lượt, không lấy trung bình trượt.

In [6]:
```
window = 100
def rolling(key):
    values = np.array([h[key] for h in history], dtype=float)
    return np.convolve(values, np.ones(window) / window, mode="valid")

x = np.arange(window, EPISODES + 1)
fig, axes = plt.subplots(2, 2, figsize=(11, 7), constrained_layout=True)
for ax, key, title, color in zip(
    axes.flat[:3], ["reward", "steps", "success"],
    ["Return (rolling 100)", "Steps (rolling 100)", "Training success (rolling
100)"],
    ["#2563eb", "#7c3aed", "#059669"]):
    ax.plot(x, rolling(key), color=color)
    ax.set_title(title); ax.set_xlabel("Episode"); ax.grid(alpha=0.25)
axes[0, 0].set_ylabel("Episode return")
axes[0, 1].set_ylabel("Number of actions")
axes[1, 0].set_ylabel("Success fraction"); axes[1, 0].set_ylim(-0.03, 1.03)
axes[1, 1].plot([h["episode"] for h in history],
                [h["epsilon"] for h in history], color="#d97706")
axes[1, 1].set(title="Exploration schedule", xlabel="Episode",
ylabel="Epsilon")
axes[1, 1].grid(alpha=0.25)
fig.savefig(ART / "learning_curves.png", dpi=200)
plt.show()
for label, block in [("First 500", history[:500]), ("Last 500",
history[-500:])]:
    print(f"{label}: mean return={np.mean([h['reward'] for h in block]):.2f},
"
          f"success={np.mean([h['success'] for h in block]):.1%}")
```

```
First 500: mean return=-20.10, success=0.0%
Last 500: mean return=20.94, success=87.6%
```

Hình 2.8: Code vẽ đường học và output so sánh 500 episode đầu với 500 episode cuối.



Hình 2.9: Reward, số bước, tỷ lệ thành công trong train và lịch giảm khám phá.

Quan sát. Reward trung bình 500 lượt đầu là -20.098, chưa có lượt tới đích. 500 lượt cuối có reward trung bình 20.940 và tỷ lệ thành công 87.6%. Trên đồ thị, reward giảm ở giai đoạn đầu rồi tăng mạnh khoảng episode 600–1.000.

Diễn giải. Ban đầu, robot kết thúc sớm do va chạm. Khi đi được xa hơn nhưng chưa tới đích, số bước tăng và hình phạt tích lũy cũng tăng nên reward có thể giảm. Sau khi tìm được đường tới đích, thưởng $+50$ làm tổng reward tăng. Vì vậy, không thể đọc việc reward giảm ban đầu thành kết luận thuật toán hoàn toàn không học.

Ảnh hưởng của khám phá. Train vẫn giữ $\epsilon = 0.01$, nên đôi lúc robot thử hành động không tốt và đâm tường. Đường thành công dao động dưới 100% phù hợp với cơ chế này. Cần đánh giá riêng chính sách greedy để biết bảng Q cuối cùng thực hiện được gì khi tắt khám phá.

2.7 Bước 7 Đánh giá và đối chiếu đường ngắn nhất

Chương trình đánh giá ba chính sách trên cùng bản đồ, cùng điểm đầu và cùng giới hạn bước: greedy với Q chưa train, chọn ngẫu nhiên và greedy sau train. Mỗi chính sách chạy 1.000 lượt. Bộ sinh ngẫu nhiên của đánh giá dùng các seed từ 10.000 đến 10.999.

In [7]:
```
def shortest_path(initial=start):
    queue = deque([(initial, [initial])])
    seen = {initial}
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path
        for dr, dc in actions:
            nxt = (state[0] + dr, state[1] + dc)
            if valid(nxt) and nxt not in seen:
                seen.add(nxt)
                queue.append((nxt, path + [nxt]))
    return []

optimal = shortest_path()
path, trace, total, status = rollout(q)
results = []
print("Policy                Success     Mean return    Mean steps")
for name, table, random_policy in [
    ("Untrained greedy", q_initial, False),
    ("Random", q_initial, True),
    ("Trained greedy", q, False)]:
    runs = [rollout(table, random_policy, 10000 + i) for i in range(1000)]
    row = {"policy": name, "episodes": 1000,
           "success_rate": float(np.mean([r[3] == "goal" for r in runs])),
           "mean_return": float(np.mean([r[2] for r in runs])),
           "mean_steps": float(np.mean([len(r[1]) for r in runs]))}
    results.append(row)
    print(f"{name:<22} {row['success_rate']:7.1%} "
          f"{row['mean_return']:15.3f} {row['mean_steps']:13.3f}")
print(f"BFS shortest path: {len(optimal)-1} actions")
print(f"Trained path: {len(trace)} actions, return={total}, terminal=
{status}")
assert status == "goal"
assert len(trace) == len(optimal) - 1
assert all(valid(s) for s in path)
assert total == 50 - (len(trace) - 1)
print("Checks passed: goal reached, valid path, BFS length, reward total.")
```

Hình 2.10: Mã đánh giá và BFS chỉ dùng để đối chiếu sau train.

Hàm BFS tìm đường ngắn nhất trên lưới bằng cách duyệt các ô hợp lệ theo từng lớp khoảng cách. Kết quả chỉ được dùng sau train. Các câu lệnh `assert` kiểm tra robot đã tới đích, mọi vị trí đều hợp lệ, số bước bằng BFS và reward khớp số hành động.

2.7.1 Output của phép đánh giá

```
Policy                Success     Mean return    Mean steps
Untrained greedy          0.0%         -10.000         1.000
Random                    0.0%         -10.530         1.530
Trained greedy          100.0%          27.000        24.000
BFS shortest path: 24 actions
Trained path: 24 actions, return=27, terminal=goal
Checks passed: goal reached, valid path, BFS length, reward total.
```

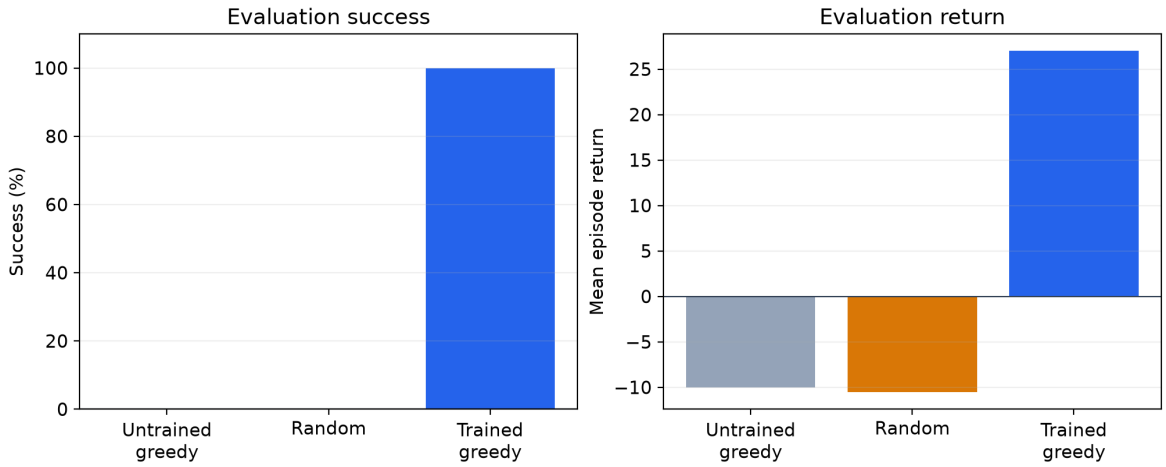
Hình 2.11: Output cell 7, gồm ba chính sách và kết quả các phép kiểm tra.

Bảng 2.1: Đánh giá 1.000 lượt cho mỗi chính sách từ $(0, 0)$.

Chính sách	Tới đích	Reward trung bình	Số bước trung bình
Greedy chưa train	0.0%	-10.000	1.000
Ngẫu nhiên	0.0%	-10.530	1.530
Greedy sau train	100.0%	27.000	24.000

Q ban đầu bằng nhau nên `argmax` chọn hành động đầu tiên là trái. Từ $(0, 0)$, hành động này đi ra ngoài và kết thúc ngay. Chính sách ngẫu nhiên cũng thường thất bại sớm. Số bước thấp của hai chính sách này vì thế không có ý nghĩa tối ưu đường đi.

2.7.2 Đọc kết quả đánh giá cùng các điều kiện chạy



Hình 2.12: So sánh tỷ lệ tới đích và tổng reward của ba chính sách.

Đường greedy sau train có 24 hành động, đúng bằng đường ngắn nhất tìm được bằng BFS. Robot nhận 23 lần reward -1 và một lần $+50$, nên:

$$R = 23 \times (-1) + 50 = 27.$$

Số 24 là số hành động; danh sách đường đi có 25 vị trí vì chứa cả vị trí ban đầu. Phân biệt hai cách đếm này tránh sai lệch một bước khi đối chiếu ảnh demo với log.

Tỷ lệ 100% ở đây có điều kiện cụ thể: bản đồ cố định, xuất phát tại $(0, 0)$, chuyển động tất định và không khám phá. Với cùng Q và cùng điểm đầu, greedy tạo ra cùng chuỗi hành động trong cả 1.000 lượt. Con số này xác nhận đường đi hoạt động ổn định trong cấu hình đang xét, không phải bằng chứng robot giải được 1.000 mê cung khác nhau.

Trong bài toán này, phạt mỗi bước khiến việc đi vòng giảm tổng reward không chiết khấu. Tuy nhiên, hàm Q được học với $\gamma = 0.9$, còn bảng đánh giá ghi tổng reward không chiết khấu. Hai đại lượng có vai trò khác nhau: Q điều khiển lựa chọn hành động; tổng reward dùng để mô tả lượt chạy thực tế. Không lấy trực tiếp một ô Q làm tổng điểm demo.

2.8 Bước 8 Kiểm tra seed và các điểm xuất phát khác

Để kiểm tra kết quả có phụ thuộc riêng seed 42 hay không, chương trình train lại từ Q bằng 0 với năm seed: 0, 7, 21, 42 và 99. Mỗi lần vẫn dùng 5.000 episode cùng bộ siêu tham số. Bảng Q của lượt train chính được giữ riêng cho demo và phép kiểm tra điểm đầu.

```
In [8]: seed_rows = []
        print("Seed   Status       Actions   Return   Gap to BFS")
        for seed in [0, 7, 21, 42, 99]:
            q_seed, _ = train(seed)
            p, tr, ret, end = rollout(q_seed)
            gap = len(tr) - (len(optimal) - 1) if end == "goal" else None
            seed_rows.append({"seed": seed, "status": end, "steps": len(tr),
                              "return": ret, "gap_to_bfs": gap})
            print(f"{seed:4d}   {end:<10} {len(tr):7d} {ret:8d}   {gap}")
        starts = [tuple(map(int, s)) for s in np.argwhere(maze == 0)
                  if tuple(s) != goal]
        start_runs = [rollout(q, initial=s) for s in starts]
        start_successes = sum(r[3] == "goal" for r in start_runs)
        failed_starts = [s for s, r in zip(starts, start_runs) if r[3] != "goal"]
        print(f"All free starts: {start_successes}/{len(starts)} reached the goal")
        print(f"Failed starts: {failed_starts}")
```

```
Seed   Status       Actions   Return   Gap to BFS
   0   goal            24       27   0
   7   goal            24       27   0
  21   goal            24       27   0
  42   goal            24       27   0
  99   goal            24       27   0
All free starts: 27/42 reached the goal
Failed starts: [(3, 1), (4, 1), (5, 1), (5, 3), (5, 4), (5, 5), (6, 1), (6, 3),
(7, 1), (7, 3), (7, 5), (8, 1), (8, 3), (8, 5), (9, 3)]
```

Hình 2.13: Code kiểm tra năm seed và output thử tất cả ô xuất phát hợp lệ.

Cả năm seed đều tới đích sau 24 hành động, nhận reward 27 và không lệch độ dài so với BFS. Kết quả này hỗ trợ nhận xét rằng nghiệm từ $(0, 0)$ không chỉ xuất hiện ở một seed đã chọn. Nó vẫn là kiểm tra trên cùng bản đồ và cùng lịch huấn luyện.

Khi đổi điểm xuất phát sang từng ô trống không phải đích, chính sách seed 42 chỉ tới đích từ 27 trong 42 ô, tương đương 64.3%. Có 15 ô thất bại. Huấn luyện luôn bắt đầu ở $(0, 0)$ nên trải nghiệm tập trung vào các vùng nằm trên hoặc gần đường thường đi; các nhánh ít được khám phá có thể còn giá trị Q chưa đủ chính xác.

Do đó, có thể kết luận robot đã học được đường tối ưu từ điểm đầu được giao, nhưng chưa có một chính sách tốt cho mọi ô trống. Nếu nhiệm vụ yêu cầu xuất phát ở vị trí bất kỳ, cần đổi cách lấy trạng thái ban đầu trong train và đánh giá lại. Không điều chỉnh lại thí nghiệm để xóa những thất bại này khỏi kết quả.

2.9 Bước 9 Demo tự hoạt động sau train

Cell demo nạp lại `artifacts/q_table.npy`. Mỗi lượt robot chọn `argmax Q(state)`, gọi `step`, ghi kết quả rồi tiếp tục cho tới khi kết thúc. Không có lệnh train lại hoặc gọi BFS trong quá trình này. Câu lệnh so sánh hai mảng xác nhận bảng đã nạp đúng với bảng vừa lưu.

In [9]:
```
q_saved = np.load(ART / "q_table.npy", allow_pickle=False)
demo_path, demo_trace, demo_return, demo_status = rollout(q_saved)
assert np.array_equal(q_saved, q)
assert demo_status == "goal"
print(f"Reloaded Q: {q_saved.shape}")
print("Step | State    | Action | Next     | Reward | Status")
for t, s, a, nxt, r, end in demo_trace:
    print(f"{t:4d} | {str(s):8} | {a:6} | {str(nxt):8} | {r:6d} | {end}")
print(f"Demo finished: {demo_status}, actions={len(demo_trace)}, return=
{demo_return}")

indices = np.linspace(0, len(demo_path) - 1, 6, dtype=int)
fig, axes = plt.subplots(2, 3, figsize=(12, 8), constrained_layout=True)
for ax, i in zip(axes.flat, indices):
    draw_maze(ax, demo_path[:i+1], f"Step {i:02d} | position {demo_path[i]}")
fig.savefig(ART / "demo_contact_sheet.png", dpi=200)
plt.show()
```

Hình 2.14: Mã demo dùng bảng Q đã lưu và lấy các khung hình từ rollout.

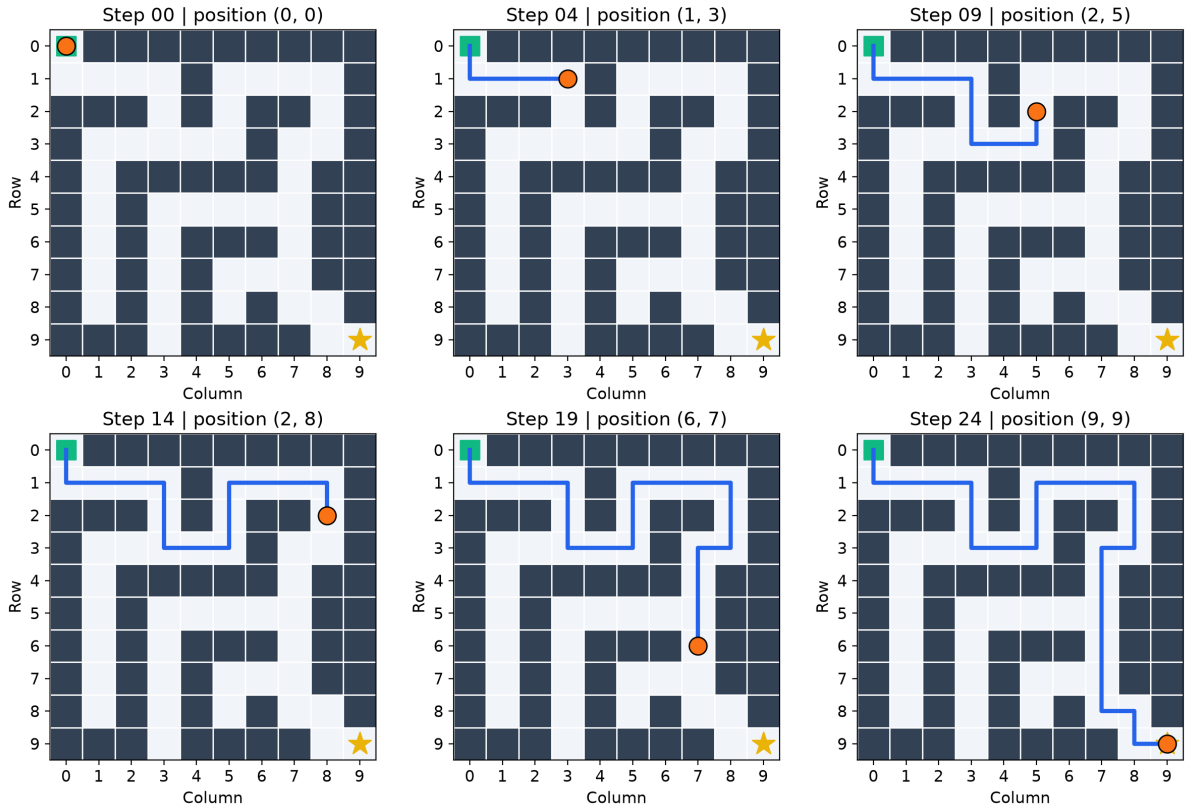
2.9.1 Output từng hành động của tác tử

```
Reloaded Q: (10, 10, 4)
Step | State    | Action | Next     | Reward | Status
   1 | (0, 0)   | DOWN   | (1, 0)   |     -1 | running
   2 | (1, 0)   | RIGHT  | (1, 1)   |     -1 | running
   3 | (1, 1)   | RIGHT  | (1, 2)   |     -1 | running
   4 | (1, 2)   | RIGHT  | (1, 3)   |     -1 | running
   5 | (1, 3)   | DOWN   | (2, 3)   |     -1 | running
   6 | (2, 3)   | DOWN   | (3, 3)   |     -1 | running
   7 | (3, 3)   | RIGHT  | (3, 4)   |     -1 | running
   8 | (3, 4)   | RIGHT  | (3, 5)   |     -1 | running
   9 | (3, 5)   | UP     | (2, 5)   |     -1 | running
  10 | (2, 5)   | UP     | (1, 5)   |     -1 | running
  11 | (1, 5)   | RIGHT  | (1, 6)   |     -1 | running
  12 | (1, 6)   | RIGHT  | (1, 7)   |     -1 | running
  13 | (1, 7)   | RIGHT  | (1, 8)   |     -1 | running
  14 | (1, 8)   | DOWN   | (2, 8)   |     -1 | running
  15 | (2, 8)   | DOWN   | (3, 8)   |     -1 | running
  16 | (3, 8)   | LEFT   | (3, 7)   |     -1 | running
  17 | (3, 7)   | DOWN   | (4, 7)   |     -1 | running
  18 | (4, 7)   | DOWN   | (5, 7)   |     -1 | running
  19 | (5, 7)   | DOWN   | (6, 7)   |     -1 | running
  20 | (6, 7)   | DOWN   | (7, 7)   |     -1 | running
  21 | (7, 7)   | DOWN   | (8, 7)   |     -1 | running
  22 | (8, 7)   | RIGHT  | (8, 8)   |     -1 | running
  23 | (8, 8)   | DOWN   | (9, 8)   |     -1 | running
  24 | (9, 8)   | RIGHT  | (9, 9)   |     50 | goal
Demo finished: goal, actions=24, return=27
```

Hình 2.15: Output đầy đủ 24 hành động khi thực thi chính sách sau train.

Log bắt đầu bằng `DOWN` từ $(0, 0)$ tới $(1, 0)$, sau đó robot đi dọc các hành lang để tới vùng bên phải bản đồ. Hành động cuối là `RIGHT` từ $(9, 8)$ tới $(9, 9)$, nhận reward $+50$ và trạng thái `goal`. Chuỗi này được dùng trực tiếp để tạo ảnh demo, không điền thủ công một đường tìm được bằng thuật toán khác.

2.9.2 Các thời điểm trong cùng một lượt tự chạy



Hình 2.16: Sáu khung hình từ rollout sau train: bước 0, 4, 9, 14, 19 và 24.

Đường xanh chỉ phần hành trình đã thực hiện đến thời điểm của từng khung hình. Chấm cam là vị trí hiện tại của robot. Ở bước 9, robot ở $(2, 5)$; bước 14 ở $(2, 8)$; bước 19 ở $(6, 7)$. Khung cuối ghi vị trí $(9, 9)$, khớp với log và điều kiện kết thúc của môi trường.

Ảnh tổng hợp giúp xem trình tự trên bản in. Tệp `agent_demo.gif` đi kèm notebook hiển thị đủ 25 vị trí từ lúc bắt đầu đến khi tới đích. Có thể mở GIF để theo dõi chuyển động mà không cần chạy lại toàn bộ quá trình huấn luyện.

2.10 Bước 10 Xuất demo và lưu kết quả

2.10.1 Tạo GIF từ rollout thực tế

Danh sách `demo_path` cung cấp các vị trí liên tiếp. Vòng lặp vẽ từng đoạn đường đã đi, lấy ảnh từ canvas rồi ghép lại bằng Pillow. Các khung cuối được giữ lâu hơn để nhìn rõ trạng thái tới đích. Việc tạo GIF chỉ thể hiện lại kết quả của chính sách, không can thiệp vào hành động.

In [10]:
```
frames = []
for i in range(len(demo_path)):
    fig, ax = plt.subplots(figsize=(5.4, 5.4))
    draw_maze(ax, demo_path[:i+1], f"Trained Q-learning agent | step {i:02d}")
    fig.tight_layout()
    fig.canvas.draw()
    rgba = np.asarray(fig.canvas.buffer_rgba())
    frames.append(Image.fromarray(rgba.copy()).convert("RGB"))
    plt.close(fig)
frames[0].save(ART / "agent_demo.gif", save_all=True,
               append_images=frames[1:] + [frames[-1]] * 6,
               duration=180, loop=0)
display(HTML('<img src="artifacts/agent_demo.gif" alt="Agent after training"
width="500">'))
print(f"Saved GIF from {len(demo_path)} rollout positions.")
```

```
Saved GIF from 25 rollout positions.
```

Hình 2.17: Code xuất GIF và output số vị trí lấy từ rollout.

2.10.2 Vẽ biểu đồ so sánh chính sách

In [11]:
```
fig, axes = plt.subplots(1, 2, figsize=(10, 4), constrained_layout=True)
names = [r["policy"].replace(" ", "\n") for r in results]
colors = ["#94a3b8", "#d97706", "#2563eb"]
axes[0].bar(names, [100*r["success_rate"] for r in results], color=colors)
axes[0].set(title="Evaluation success", ylabel="Success (%)", ylim=(0, 110))
axes[1].bar(names, [r["mean_return"] for r in results], color=colors)
axes[1].set(title="Evaluation return", ylabel="Mean episode return")
for ax in axes:
    ax.axhline(0, color="#334155", linewidth=0.8)
    ax.grid(axis="y", alpha=0.2)
fig.savefig(ART / "policy_comparison.png", dpi=200)
plt.show()
print("Saved: artifacts/policy_comparison.png")
```

```
Saved: artifacts/policy_comparison.png
```

Hình 2.18: Cell tạo biểu đồ từ bảng kết quả đánh giá và output lưu ảnh.

Hai biểu đồ sử dụng đúng các giá trị trong danh sách `results` của Bước 7. Trục trái là phần trăm lượt tới đích; trục phải là tổng reward trung bình. Hình so sánh đã được trình bày cùng phần phân tích để đọc số liệu và nguyên nhân chênh lệch trong cùng ngữ cảnh.

2.10.3 Lưu log, cấu hình và số liệu

In [12]:
```
with (ART / "demo_trace.csv").open("w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["step", "state", "action", "next_state", "reward",
"status"])
    writer.writerows(demo_trace)
summary = {"config": {"seed": SEED, "episodes": EPISODES, "alpha": ALPHA,
           "gamma": GAMMA, "epsilon_start": EPS_START, "epsilon_min": EPS_MIN,
           "epsilon_decay": EPS_DECAY, "max_steps": MAX_STEPS},
           "versions": {"python": platform.python_version(), "numpy":
np.__version__,
                        "matplotlib": matplotlib.__version__},
           "evaluation": results, "seed_checks": seed_rows,
           "bfs_steps": len(optimal)-1, "demo_steps": len(demo_trace),
           "demo_return": demo_return, "demo_status": demo_status,
           "demo_path": demo_path, "all_starts_total": len(starts),
           "all_starts_success": start_successes, "failed_starts":
failed_starts,
           "first500_return": float(np.mean([h["reward"] for h in
history[:500]])),
           "last500_return": float(np.mean([h["reward"] for h in
history[-500:]])),
           "first500_success": float(np.mean([h["success"] for h in
history[:500]])),
           "last500_success": float(np.mean([h["success"] for h in
history[-500:]]))}
(ART / "results.json").write_text(json.dumps(summary, indent=2),
encoding="utf-8")
print("Saved: agent_demo.gif, policy_comparison.png, demo_trace.csv,
results.json")
```

```
Saved: agent_demo.gif, policy_comparison.png, demo_trace.csv, results.json
```

Hình 2.19: Cell xuất CSV, JSON và output xác nhận các tệp kết quả.

`training_history.csv` lưu 5.000 dòng log train; `demo_trace.csv` lưu 24 hành động demo. `results.json` giữ cấu hình, phiên bản thư viện, kết quả đánh giá và các trường hợp thất bại để đối chiếu với báo cáo.

2.10.4 Chạy lại và chuẩn bị đưa lên GitHub

Thư mục bàn giao có notebook đã chạy, `requirements.txt`, `README.md`, artifact và nguồn LaTeX. Cài thư viện bằng `pip install -r requirements.txt`, mở notebook và chọn `Restart Kernel / Run All` từ thư mục chứa notebook.

Khi đưa lên GitHub, giữ notebook cùng các ảnh, GIF, CSV, JSON và bảng Q trong `artifacts/`. Tệp `.gitignore` loại môi trường Python và tệp tạm LaTeX. Notebook giữ sẵn output để đọc trước khi chạy lại.

Tài liệu tham khảo

[1] Google Cloud. *What are AI agents? Definition, examples, and types.* https://cloud.google.com/discover/what-are-ai-agents. Truy cập ngày 23/10/2026.

[2] Anna Gutowska, IBM. *What are AI agents?* https://www.ibm.com/think/topics/ai-agents. Truy cập ngày 23/10/2026.

[3] McKinsey. *What is an AI agent?* 25/03/2025. https://www.mckinsey.com/featured-insights/mckinsey-explainers/what-is-an-ai-agent. Truy cập ngày 23/10/2026.

[4] GeeksforGeeks. *Reinforcement Learning.* https://www.geeksforgeeks.org/machine-learning/what-is-reinforcement-learning/. Truy cập ngày 23/10/2026. Nguồn bài toán và bản đồ mê cung; mã thực nghiệm được viết lại trong notebook kèm theo.

[5] Shunyu Yao và cộng sự. *ReAct: Synergizing Reasoning and Acting in Language Models.* ICLR 2023. https://arxiv.org/abs/2210.03629.